

Javascript atelier 12

Exercice : Navigateur 1.....2

Exercice : Navigateur 2.....7

Exercice : Navigateur 1

Exercices

Enfants DOM

Regardez cette page :

```
1 <html>
2 <body>
3   <div>Users:</div>
4   <ul>
5     <li>John</li>
6     <li>Pete</li>
7   </ul>
8 </body>
9 </html>
```

Pour chacun des éléments suivants, donnez au moins un moyen d'y accéder :

- Le nœud `<div>` du DOM ?
- Le nœud `<ul>` du DOM ?
- Le deuxième `<li>` (avec Pete) ?

L'objectif était d'accéder à des nœuds spécifiques à partir d'une structure HTML donnée.

Le nœud `<div>` : On y accède via `document.body.firstChild` car il est le premier élément enfant du corps.

Le nœud `<ul>` : On y accède via `document.body.children[1]` (le deuxième élément enfant) ou avec `document.querySelector('ul')`.

Le deuxième `<li>` (Pete) : On utilise `document.querySelectorAll('li')[1]` pour cibler le deuxième élément de la liste des puces.

La question des frères et sœurs

Si `element` – est un nœud élément arbitraire du DOM ...

- Est-il vrai que `elem.lastChild.nextSibling` est toujours `null` ?
- Est-il vrai que `elem.children[0].previousSibling` est toujours `null` ?

Analyse des propriétés de navigation entre les nœuds.

`elem.lastChild.nextSibling` est-il toujours `null` ?

Vrai : Par définition, le dernier enfant (`lastChild`) n'a aucun frère qui le suit (`nextSibling`).

elem.children[0].previousSibling est-il toujours null ?

Faux : children[0] est le premier élément (balise), mais previousSibling cherche le premier nœud. S'il y a un espace ou un retour à la ligne avant la balise dans le code HTML, ce nœud de texte sera renvoyé à la place de null.

Sélectionner toutes les cellules diagonales

Écrivez le code pour colorer toutes les cellules du tableau diagonal en rouge.

Vous devrez obtenir toutes les diagonales `<td>` de la `<table>` et les colorer en utilisant le code :

```
1 // td doit être la référence à la cellule du tableau
2 td.style.backgroundColor = 'red';
```

Le résultat devrait être :

1:1	2:1	3:1	4:1	5:1
1:2	2:2	3:2	4:2	5:2
1:3	2:3	3:3	4:3	5:3
1:4	2:4	3:4	4:4	5:4
1:5	2:5	3:5	4:5	5:5

Algorithme pour colorer la diagonale d'un tableau en rouge.

```
let table = document.querySelector('table');

for (let i = 0; i < table.rows.length; i++) {
  let td = table.rows[i].cells[i];
  td.style.backgroundColor = 'red';
}
```

Explication : On utilise une boucle for pour parcourir les lignes (rows). Pour chaque ligne d'indice `$i`, on sélectionne la cellule (cells) ayant le même indice `$i`, ce qui trace visuellement la diagonale du tableau.

Exercices

Recherche d'éléments

Voici le document avec le tableau et formulaire

Comment trouver ?...

1. Le tableau avec `id="age-table"`.
2. Tous les éléments `label` dans ce tableau (il devrait y en avoir 3).
3. Le premier `td` dans ce tableau (avec le mot "Age").
4. Le `form` avec `name="search"`.
5. Le premier `input` dans ce formulaire.
6. Le dernier `input` dans ce formulaire.

Ouvrez la page [table.html](#) dans un onglet à part et utilisez les outils du navigateur pour cela.

1. Navigation dans l'arborescence (Enfants DOM)

L'objectif est d'accéder aux éléments d'une page HTML simple sans utiliser d'ID.

Le nœud `<div>` : On utilise `document.body.firstChild` car c'est le tout premier élément dans le corps du document.

Le nœud `<ul>` : On utilise `document.body.children[1]` pour cibler le deuxième élément du `body`.

Le deuxième `<li>` (Pete) : On utilise `document.querySelectorAll('li')[1]` pour récupérer le deuxième élément de la liste.

2. Logique des relations (Frères et sœurs)

Analyse des propriétés de parenté entre les nœuds.

`elem.lastChild.nextSibling` est toujours null ?

VRAI : Le dernier enfant n'a, par définition, aucun frère après lui.

`elem.children[0].previousSibling` est toujours null ?

FAUX : `children[0]` est le premier élément (balise), mais il peut y avoir un nœud de texte (un espace ou un retour à la ligne) juste avant lui. Dans ce cas, `previousSibling` renverra ce texte et non null.

3. Sélection des cellules diagonales

Algorithme pour colorer automatiquement la diagonale d'un tableau.

```
let table = document.querySelector('table');  
  
for (let i = 0; i < table.rows.length; i++) {  
  table.rows[i].cells[i].style.backgroundColor = 'red';  
}
```

La boucle parcourt chaque ligne (\$i\$). Pour la ligne 0, elle colore la cellule 0 ; pour la ligne 1, la cellule 1, créant ainsi une ligne diagonale rouge.

4. Recherche d'éléments complexes (Fichier table.html)

Utilisation des méthodes de recherche sur une structure réelle.

Tableau par ID : `document.getElementById('age-table')`.

Tous les labels du tableau : `table.querySelectorAll('label')`.

Premier TD (Age) : `table.querySelector('td')`.

Formulaire "search" : `document.querySelector('form[name="search"]')`.

Premier Input du formulaire : `form.querySelector('input')`.

Dernier Input du formulaire : On récupère tous les inputs et on prend le dernier index : `inputs[inputs.length - 1]`.

Compter les descendants

Qu'affiche le script ?

```
1 <html>
2
3 <body>
4   <script>
5     alert(document.body.lastChild.nodeType);
6   </script>
7 </body>
8
9 </html>
```

1. Compter les descendants : Qu'affiche le script ?

Dans l'image avec le code `<script>`, on demande la valeur de `document.body.lastChild.nodeType`.

Le script affiche 1.

Pourquoi ? * Au moment où le script s'exécute, il est lui-même le dernier enfant du `<body>` (car le navigateur lit le code de haut en bas).

`lastChild` correspond donc à la balise `<script>`.

Le `nodeType` pour un élément HTML (une balise) est toujours 1.

Balise dans le commentaire

Qu'affiche ce code ?

```
1 <script>
2   let body = document.body;
3
4   body.innerHTML = "<!--" + body.tagName + "-->";
5
6   alert( body.firstChild.data ); // Qu'est ce qu'il y a ici ?
7 </script>
```

Où est le "document" dans la hiérarchie ?

À quelle classe appartient le `document` ?

Quelle est sa place dans la hiérarchie DOM ?

Hérite-t-il de `Node` ou `Element`, ou peut-être de `HTMLElement` ?

1. Compter les descendants

Question : Qu'affiche le script ?

Réponse : Il affiche 1.

Pourquoi ?

Le code demande le `nodeType` du dernier enfant du `<body>` au moment où le script s'exécute.

Pendant que le navigateur lit le HTML, il s'arrête dès qu'il rencontre la balise `<script>` pour l'exécuter.

À cet instant précis, le dernier élément inséré dans le `<body>` est la balise `<script>` elle-même.

Comme une balise (un élément HTML) a un `nodeType` de valeur 1, l'alerte affiche 1.

2. Balise dans le commentaire

Question : Qu'affiche ce code ?

Réponse : Il affiche BODY.

Pourquoi ?

`body.tagName` vaut "BODY" (en majuscules par convention en HTML).

La ligne `body.innerHTML = ""`; remplace tout le contenu du body par un commentaire HTML : ``.

Le `body.firstChild` devient donc ce nœud de commentaire.

La propriété `.data` sur un nœud de commentaire permet d'extraire le texte à l'intérieur du commentaire.

L'alerte affiche donc le contenu textuel du commentaire, soit BODY.

1. À quelle classe appartient le document ? Il appartient à la classe `HTMLDocument` (ou `Document` selon les normes). C'est l'objet global qui représente toute la page chargée dans le navigateur.

2. Quelle est sa place dans la hiérarchie DOM ? C'est la racine logique (le sommet) de l'arborescence. C'est par lui que l'on passe pour accéder à n'importe quel élément de la page (via `document.body`, `document.getElementById`, etc.).

3. Hérite-t-il de Node, Element ou HTMLElement ? Il hérite de `Node`, car c'est un nœud de l'arbre. En revanche, il n'hérite pas de `Element` ni de `HTMLElement`, car ces classes sont réservées aux balises spécifiques (comme `<div>` ou `<p>`), alors que le document est le conteneur global.

Exercice : Navigateur 2

Obtenez l'attribut

Écrivez le code pour sélectionner l'élément avec l'attribut `data-widget-name` dans le document et pour lire sa valeur.

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5   <div data-widget-name="menu">Choose the genre</div>
6
7   <script>
8     /* your code */
9   </script>
10 </body>
11 </html>
```

```
1 let widget = document.querySelector('[data-widget-name]');
2 let name = widget.dataset.widgetName;
3 console.log(name);

let widget = document.querySelector('[data-widget-name]');
let name = widget.getAttribute('data-widget-name');
console.log(name);
```

Rendre les liens externes orange

Mettez tous les liens externes en orange en modifiant leur propriété `style`.

Un lien est externe si :

- Son `href` contient `://`
- Mais ne commence pas par `http://internal.com`.

Exemple :

```
1 <a name="list">the list</a>
2 <ul>
3   <li><a href="http://google.com">http://google.com</a></li>
4   <li><a href="/tutorial">/tutorial.html</a></li>
5   <li><a href="local/path">local/path</a></li>
6   <li><a href="ftp://ftp.com/my.zip">ftp://ftp.com/my.zip</a></li>
7   <li><a href="http://nodejs.org">http://nodejs.org</a></li>
8   <li><a href="http://internal.com/test">http://internal.com/test</a></li>
9 </ul>
10
11 <script>
12   // setting style for a single link
13   let link = document.querySelector('a');
14   link.style.color = 'orange';
15 </script>
```

Le résultat devrait être :

The list:

- <http://google.com>
- </tutorial.html>
- <local/path>
- <ftp://ftp.com/my.zip>
- <http://nodejs.org>
- <http://internal.com-test>

```
let links = document.querySelectorAll('a');

for (let link of links) {
  let href = link.getAttribute('href');
  if (href && href.includes('://') && !href.startsWith('http://internal.com')) {
    link.style.color = 'orange';
  }
}
```

createTextNode vs innerHTML vs textContent

Nous avons un élément DOM vide `elem` et une chaîne de caractères `text`.

Lesquelles de ces 3 commandes feront exactement la même chose ?

1. `elem.append(document.createTextNode(text))`
2. `elem.innerHTML = text`
3. `elem.textContent = text`

Effacer l'élément

1. `elem.append(document.createTextNode(text))`

Cette méthode crée explicitement un nœud de texte.

Tous les caractères contenus dans la variable `text` (y compris les balises HTML comme `<b>` ou `<script>`) sont traités comme du texte brut.

Résultat : Le texte s'affiche tel quel à l'écran, sans être interprété par le navigateur.

2. `elem.innerHTML = text`

Cette commande est l'exception du groupe.

Elle demande au navigateur d'interpréter la chaîne `text` comme du code HTML.

Si `text` contient `<b>Salut</b>`, l'utilisateur verra "Salut" en gras.

Risque : C'est une faille de sécurité potentielle (injection XSS) si la variable `text` provient d'une source non fiable.

3. `elem.textContent = text`

Cette méthode est la version moderne et simplifiée de la première option.

Elle définit le contenu d'un élément en tant que texte pur.

Comme `createTextNode`, elle ne traite pas les balises HTML et neutralise tout code malveillant.

Résultat : C'est fonctionnellement identique à l'option 1.

Effacer l'élément

Créez une fonction `clear(elem)` qui supprime tout de l'élément.

```
1 <ol id="elem">
2   <li>Hello</li>
3   <li>World</li>
4 </ol>
5
6 <script>
7   function clear(elem) { /* votre code */ }
8
9   clear(elem); // efface la liste
10 </script>
```

```
function clear(elem) {
  elem.replaceChildren();
}
```


Pourquoi "aaa" reste-t-il ?

Dans l'exemple ci-dessous, l'appel `table.remove()` supprime le tableau du document.

mais si vous l'exécutez, vous pouvez voir que le texte "aaa" est toujours visible.

Pourquoi cela se produit-il ?

```
1 <table id="table">
2   aaa
3   <tr>
4     <td>Test</td>
5   </tr>
6 </table>
7
8 <script>
9   alert(table); // la table, comme il se doit
10
11   table.remove();
12   // pourquoi y a-t-il encore "aaa" dans le document ?
13 </script>
```

1: Effacer l'élément

L'objectif est de vider tout le contenu d'un élément HTML. Voici les trois approches conceptuelles pour y parvenir :

L'approche par réinitialisation : On demande au navigateur de vider le contenu interne en lui passant une chaîne de caractères vide. C'est la méthode la plus directe pour supprimer d'un coup tous les nœuds enfants (texte, balises, commentaires).

L'approche par remplacement : On utilise une fonction moderne qui remplace tous les enfants actuels par une liste vide. Cette méthode est plus propre car elle ne nécessite pas de manipuler des chaînes de caractères HTML.

L'approche par itération (boucle) : On parcourt l'élément et on retire ses enfants un par un. Pour que cela fonctionne, il faut toujours cibler le premier élément restant jusqu'à ce qu'il n'en reste plus aucun, car la liste des enfants se met à jour en temps réel à chaque suppression.

2 : Pourquoi "aaa" reste-t-il ?

Ce problème illustre la manière dont les navigateurs gèrent les erreurs de syntaxe HTML.

L'erreur de structure : Dans le langage HTML, un tableau (`<table>`) a une structure très stricte. Il ne peut contenir que des éléments spécifiques comme des lignes ou des cellules. Le texte "aaa" a été placé "nu" à l'intérieur du tableau, ce qui est interdit par les règles du DOM.

L'auto-correction (Hoisting) : Pour éviter de casser l'affichage, le navigateur répare automatiquement le code pendant qu'il construit la page. Il extrait le texte mal placé et le remonte juste avant la balise du tableau dans l'arborescence du document.

Le résultat du script : Lorsque la commande de suppression du tableau est exécutée, elle ne cible que l'élément tableau lui-même. Comme le texte "aaa" a été déplacé à l'extérieur par le navigateur avant même que le script ne commence, il n'appartient plus au tableau et reste donc affiché sur la page.

Create a notification

Write a function `showNotification(options)` that creates a notification: `<div class="notification">` with the given content. The notification should automatically disappear after 1.5 seconds.

The options are:

```
1 // shows an element with the text "Hello" near the right-top of the window
2 showNotification({
3   top: 10, // 10px from the top of the window (by default 0px)
4   right: 10, // 10px from the right edge of the window (by default 0px)
5   html: "Hello!", // the HTML of notification
6   className: "welcome" // an additional class for the div (optional)
7 });
```

Notification is on the right

Hello 6

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Dolorum aspernatur quam ex eaque inventore quod voluptatem adipisci omnis nemo nulla fugit iste numquam ducimus cumque minima porro ea quidem maxime necessitatibus beatae labore soluta voluptatum magnam consequatur sit laboriosam velit excepturi laborum sequi eos placeat et quia deleniti? Corrupti velit impedit autem et obcaecati fuga debitis nemo ratione iste veniam amet dicta hic ipsam unde cupiditate incidunt aut iure ipsum officiis soluta temporibus. Tempore dicta ullam delectus numquam consectetur quisquam explicabo culpa excepturi placeat quo sequi molestias reprehenderit hic at nemo cumque voluptates quidem repellendus maiores unde earum molestiae ad.

Use CSS positioning to show the element at given top/right coordinates. The source document has the necessary styles.

Création d'une notification dynamique

L'objectif est de concevoir une fonction capable de générer une alerte visuelle temporaire à un endroit précis de l'écran.

1. La création de l'élément (Structure)

Pour générer la notification, on crée un nouvel élément de type "division" (div) directement en mémoire. On lui applique ensuite une classe spécifique pour hériter des styles CSS prédéfinis (couleurs, bordures, ombres). Le contenu de la notification est injecté dynamiquement afin de pouvoir afficher soit du texte simple, soit des éléments HTML plus complexes.

2. Le positionnement (Style)

La notification doit apparaître à des coordonnées précises. Pour cela, on utilise le positionnement absolu ou fixe :

On définit la distance par rapport au haut et au bord droit de la fenêtre de navigation.

Ces valeurs sont récupérées depuis les paramètres de la fonction et appliquées directement aux propriétés de style de l'élément pour garantir que l'alerte ne gêne pas la lecture du contenu principal.

3. L'intégration au document

Une fois l'élément créé et stylisé, il n'est pas encore visible. Il doit être greffé à l'arbre DOM, généralement en l'ajoutant à la fin du corps du document (body). C'est à cet instant précis que la notification apparaît pour l'utilisateur.

4. La gestion du cycle de vie (Temporisation)

Une bonne interface utilisateur ne doit pas encombrer l'écran indéfiniment.

On met en place un minuteur (timer) dès l'apparition de l'élément.

À l'expiration de ce délai (ici 1,5 seconde), une action de nettoyage est déclenchée pour retirer proprement l'élément du document. Cela libère de la mémoire et nettoie l'interface visuelle sans intervention manuelle de l'utilisateur.

Quel est le défilement à partir du bas ?

La propriété `elem.scrollTop` est la taille de la partie déroulante à partir du haut. Comment obtenir la taille du défilement inférieur (appelons-le `scrollBottom`) ?

Écrivez le code qui fonctionne pour un `element` arbitraire.

P.S. Veuillez vérifier votre code: s'il n'y a pas de défilement ou que l'élément est entièrement défilé vers le bas, alors il devrait retourner `0`.

Pour trouver la distance restante avant d'atteindre le bas d'un élément (ce qu'on appelle ici `scrollBottom`), il faut soustraire ce qui est déjà "sorti" par le haut et ce qui est actuellement visible de la hauteur totale.

Voici les trois mesures à utiliser :

La hauteur totale (`scrollHeight`) : C'est la hauteur complète du contenu, même la partie que l'on ne voit pas encore parce qu'il faut défiler.

Le défilement actuel (`scrollTop`) : C'est la mesure de la partie qui a déjà disparu vers le haut en défilant.

La hauteur visible (`clientHeight`) : C'est la hauteur de la fenêtre de l'élément (ce que l'utilisateur voit réellement à l'écran).

Le `scrollBottom` est égal à la Hauteur totale de laquelle on retire le Défilement actuel et la Hauteur visible.

Si vous arrivez tout en bas, la somme de ce que vous avez défilé (`scrollTop`) et de ce que vous voyez (`clientHeight`) devient égale à la hauteur totale du contenu (`scrollHeight`). À ce moment-là, la soustraction donne bien 0, comme demandé dans l'exercice.

Quelle est la largeur de la barre de défilement ?

Écrivez le code qui renvoie la largeur d'une barre de défilement standard.

Pour Windows, il varie généralement entre `12px` et `20px`. Si le navigateur ne lui réserve pas d'espace (la barre de défilement est à moitié translucide sur le texte, cela arrive également), alors il peut s'agir de `0px`.

P.S. Le code devrait fonctionner pour tout document HTML, ne dépend pas de son contenu.

Pour calculer la largeur de la barre de défilement (`scrollbar`) en JavaScript, l'astuce classique consiste à créer un élément temporaire avec un défilement forcé et à comparer sa largeur

totale avec la largeur de son contenu interne.

```
function getScrollbarWidth() {
  const outer = document.createElement('div');
  outer.style.visibility = 'hidden';
  outer.style.overflow = 'scroll';
  outer.style.msOverflowStyle = 'scrollbar';
  document.body.appendChild(outer);

  const inner = document.createElement('div');
  outer.appendChild(inner);

  const scrollbarWidth = (outer.offsetWidth - inner.offsetWidth);

  outer.parentNode.removeChild(outer);

  return scrollbarWidth;
}

console.log("La largeur de la barre de défilement est : " + getScrollbarWidth() + "px");
```

Pour centrer la balle précisément en JavaScript, il faut calculer le centre du terrain (partie verte) et y soustraire la moitié de la taille de la balle.

```
let field = document.getElementById('field');
let ball = document.getElementById('ball');
ball.style.left = Math.round(field.clientWidth / 2 - ball.offsetWidth / 2) + 'px';
ball.style.top = Math.round(field.clientHeight / 2 - ball.offsetHeight / 2) + 'px';
```

Voici les principales différences entre `getComputedStyle(elem).width` et `elem.clientWidth` :

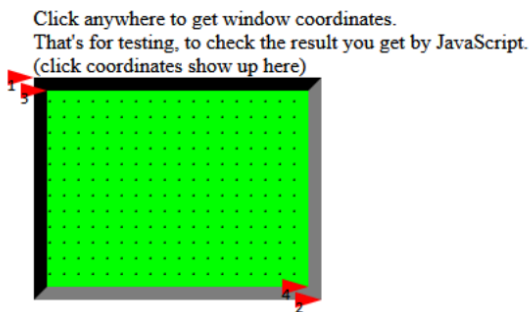
Caractéristique	<code>getComputedStyle(elem).width</code>	<code>elem.clientWidth</code>
Type de valeur	Retourne une chaîne de caractères (ex: "200px").	Retourne un nombre entier de pixels (ex: 200).
Bordures et Scrollbar	Selon box-sizing, peut inclure ou non le padding, mais n'inclut jamais la barre de défilement.	Inclut le padding mais exclut toujours la barre de défilement et les bordures.
Précision	Peut être une valeur décimale (ex: "200.5px") sur les écrans haute densité.	Est toujours arrondi à l'entier le plus proche.
Disponibilité	Peut retourner "auto" si l'élément n'est pas encore rendu ou n'a pas de largeur fixe.	Retourne toujours la dimension réelle mesurée à l'écran (ou 0 si caché).

Trouver les coordonnées de la fenêtre du champ

Dans l'iframe ci-dessous, vous pouvez voir un document avec le "champ" vert.

Utilisez JavaScript pour trouver les coordonnées de la fenêtre des coins pointés par des flèches.

Il y a une petite fonctionnalité implémentée dans le document pour plus de commodité. Un clic à n'importe quel endroit montre les coordonnées là-bas.



[champ.html](#)

Votre code doit utiliser DOM pour obtenir les coordonnées de la fenêtre de :

1. Coin extérieur supérieur gauche (c'est simple).
2. En bas à droite, coin extérieur (simple aussi).
3. Coin intérieur supérieur gauche (un peu plus dur).
4. En bas à droite, coin intérieur (il y a plusieurs façons, choisissez-en une).

Les coordonnées que vous calculez doivent être les mêmes que celles renvoyées par le clic de souris.

P.S. Le code devrait également fonctionner si l'élément a une autre taille ou bordure, qui n'est lié à aucune valeur fixe.

Pour obtenir les coordonnées par rapport à la fenêtre (viewport), nous utilisons la méthode `getBoundingClientRect()`.

```
let field = document.getElementById('field');
let rect = field.getBoundingClientRect();
let answer1 = [rect.left, rect.top];
let answer2 = [rect.right, rect.bottom];
let answer3 = [
  rect.left + field.clientLeft,
  rect.top + field.clientTop
];
let answer4 = [
  rect.left + field.clientLeft + field.clientWidth,
  rect.top + field.clientTop + field.clientHeight
];
console.log("1:", answer1, "2:", answer2, "3:", answer3, "4:", answer4);
```